MIT
MIVA OPEN UNIVERSITY

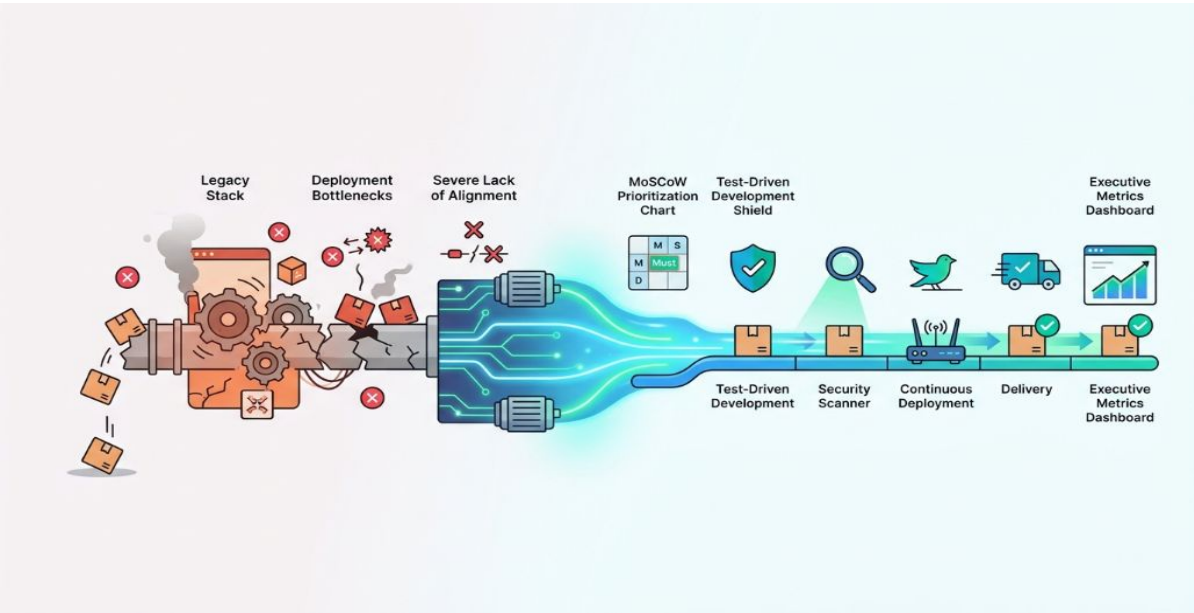
Software Development Life Cycle (SDLC)

Agile Requirements Engineering & MoSCoW Prioritisation

Project Synopsis

This laboratory centres on 'AeroFreight', a pan-African logistics and supply chain conglomerate operating major hubs in Lagos, Nairobi, Kigali, and Johannesburg. AeroFreight currently relies on a fragmented, legacy software stack built in the late 1990s. The system experiences frequent crashes, severe deployment bottlenecks, and untested code releases. Furthermore, there is a severe lack of alignment between the business stakeholders and the software engineering teams, resulting in lost packages, inaccurate tracking, and revenue leakage.

Over six integrated, self-guided laboratory sessions, the objective is to architect and deploy a modern, automated Software Development Life Cycle (SDLC) pipeline. This project will transition the engineering department into a highly disciplined, Agile-driven organisation. Moving beyond theoretical models, programmatic logic will be used to automate Agile requirements engineering via MoSCoW prioritisation, enforce architectural design governance, implement strict Test-Driven Development (TDD) protocols, build an automated Continuous Integration (CI) security scanner, deploy a Continuous Deployment (CD) Canary release router, and calculate executive-level Agile performance metrics.



© Copyright 2025 MIVA Open University All Rights Reserved

Learning Outcomes

Upon successful completion, students will be able to:

- Analyse a raw product backlog and categorise features according to the MoSCoW prioritisation framework.
- Compute total story points and determine which backlog items can fit within a defined sprint capacity.
- Apply an automated requirements engine to commit, defer, or reject backlog items based on mathematical sprint constraints.
- Simulate scope creep scenarios to evaluate the impact of additional high-priority features on sprint feasibility.
- Interpret output from the SDLC Requirements Engine to make informed decisions for sprint planning and stakeholder negotiation.

© Copyright 2025 MIVA Open University All Rights Reserved

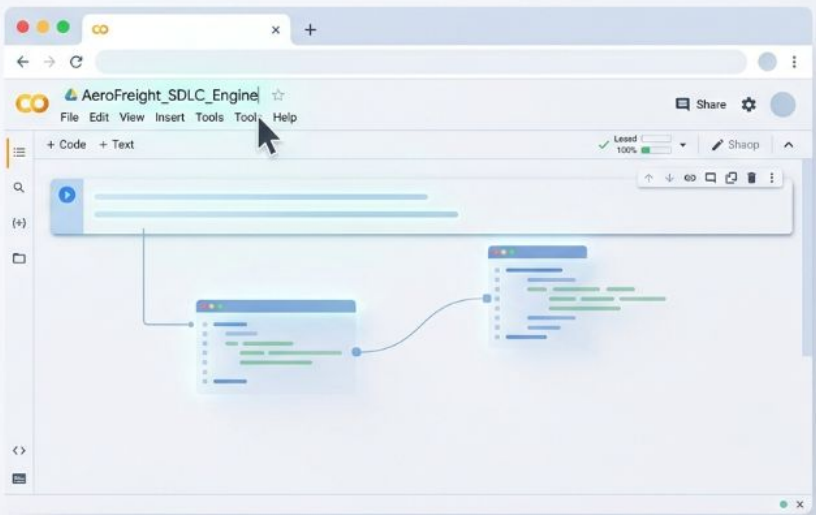
Pre-Laboratory Environment Configuration

Modern SDLC management has evolved beyond manual documentation. Today's software engineering pipelines utilise automated scripts to enforce quality, security, and velocity at scale. To simulate this programmatic governance approach, this laboratory protocol is engineered to be executed entirely within a unified computational environment.

Computational Strategy Environment Setup

Navigate to colab.research.google.com. Google Colaboratory (Colab) is a free, browser-based computational environment for Python. A new notebook must be created and utilised for all programmatic tasks across the six sessions. Full-stack software engineering expertise is not required; the provided scripts represent the automated governance logic, testing frameworks, and deployment routers used by IT management to maintain absolute control over the software lifecycle.

Create a new Python notebook and label it AeroFreight_SDLC_Engine.



© Copyright 2025 MIVA Open University All Rights Reserved

Objective

To computationally process an unstructured product backlog, automating the extraction of core business requirements using the MoSCoW prioritisation matrix to mathematically define a viable Agile Sprint.

Theoretical Framework

The first formal phase of the Software Development Life Cycle is Requirements Gathering and Analysis. In traditional Waterfall models, this involved writing hundreds of pages of rigid specifications. In modern Agile environments, requirements are captured continuously as 'User Stories' within a Product Backlog. To prevent developers from wasting time on low-value features, the MoSCoW method (Must have, Should have, Could have, Won't have) is utilised to prioritise the backlog. 'Must Have' features form the Minimum Viable Product (MVP). If a sprint is over-committed with 'Must Haves', the SDLC pipeline is mathematically guaranteed to fail before development begins.

Real-World Scenario

The AeroFreight Chief Operating Officer (COO) has submitted a massive list of features for the new logistics application. The development team in Kigali only has a capacity of 40 Story Points for the upcoming two-week Agile Sprint. The business requests a real-time GPS tracker, a predictive AI weather module, a basic login screen, and a dark mode interface, all delivered immediately. An automated Requirements Engine must be built to parse the backlog, apply MoSCoW logic, and mathematically determine what is actually possible within the sprint constraints.

© Copyright 2025 MIVA Open University All Rights Reserved

Implementation Code

Python

```
# Execute within a new Google Colab Notebook cell

# 1. Define the Raw Product Backlog from Business Stakeholders
# Format: {'StoryID': str, 'Feature': str, 'MoSCoW': str,
'StoryPoints': int}
product_backlog = [
    {'StoryID': 'US-001', 'Feature': 'Secure Driver Login', 'MoSCoW':
'Must Have', 'StoryPoints': 8},
    {'StoryID': 'US-002', 'Feature': 'Real-Time Package Tracking',
'MoSCoW': 'Must Have', 'StoryPoints': 13},
    {'StoryID': 'US-003', 'Feature': 'Payment Gateway Integration',
'MoSCoW': 'Must Have', 'StoryPoints': 13},
    {'StoryID': 'US-004', 'Feature': 'Automated SMS Notifications',
'MoSCoW': 'Should Have', 'StoryPoints': 5},
    {'StoryID': 'US-005', 'Feature': 'Dark Mode UI Toggle', 'MoSCoW':
'Could Have', 'StoryPoints': 3},
    {'StoryID': 'US-006', 'Feature': 'Predictive AI Weather Routing',
'MoSCoW': 'Wont Have', 'StoryPoints': 21}
]

# 2. Define the Agile Sprint Capacity
sprint_capacity_points = 40

# 3. Define the Requirements Prioritization Engine
def execute_moscow_sprint_planning(backlog, capacity):
    print(f"\n[SDLC REQUIREMENTS ENGINE] Initiating Sprint Planning.
Total Capacity: {capacity} Points\n")

    approved_sprint = []
    total_committed_points = 0
```

© Copyright 2025 MIVA Open University All Rights Reserved

Implementation Code

```
# Priority 1: Extract and commit the absolute Minimum Viable Product
(MVP)
    for story in backlog:
        if story['MoSCoW'] == 'Must Have':
            if (total_committed_points + story['StoryPoints']) <=
capacity:
                approved_sprint.append(story)
                total_committed_points += story['StoryPoints']
                print(f"[COMMITTED] {story['StoryID']}:
{story['Feature']} ({story['StoryPoints']} pts)")
            else:
                print(f"[WARNING] Cannot commit 'Must Have'
{story['StoryID']}. Sprint Capacity Exceeded!")
                return False, []

# Priority 2: Fill remaining capacity with 'Should Have' features
    for story in backlog:
        if story['MoSCoW'] == 'Should Have':
            if (total_committed_points + story['StoryPoints']) <=
capacity:
                approved_sprint.append(story)
                total_committed_points += story['StoryPoints']
                print(f"[COMMITTED] {story['StoryID']}:
{story['Feature']} ({story['StoryPoints']} pts)")

# Priority 3: Explicitly reject out-of-scope features
    for story in backlog:
        if story['MoSCoW'] == 'Wont Have':
            print(f"[REJECTED] {story['StoryID']}: {story['Feature']}
is explicitly out of scope for this release.")

    print(f"\n[SPRINT APPROVED] Total Points Committed:
{total_committed_points} / {capacity}")
    return True, approved_sprint

# 4. Execute the SDLC Sprint Planning
is_valid, final_sprint_backlog =
execute_moscow_sprint_planning(product_backlog, sprint_capacity_points)
```

© Copyright 2025 MIVA Open University All Rights Reserved

Line-by-Line Rationale

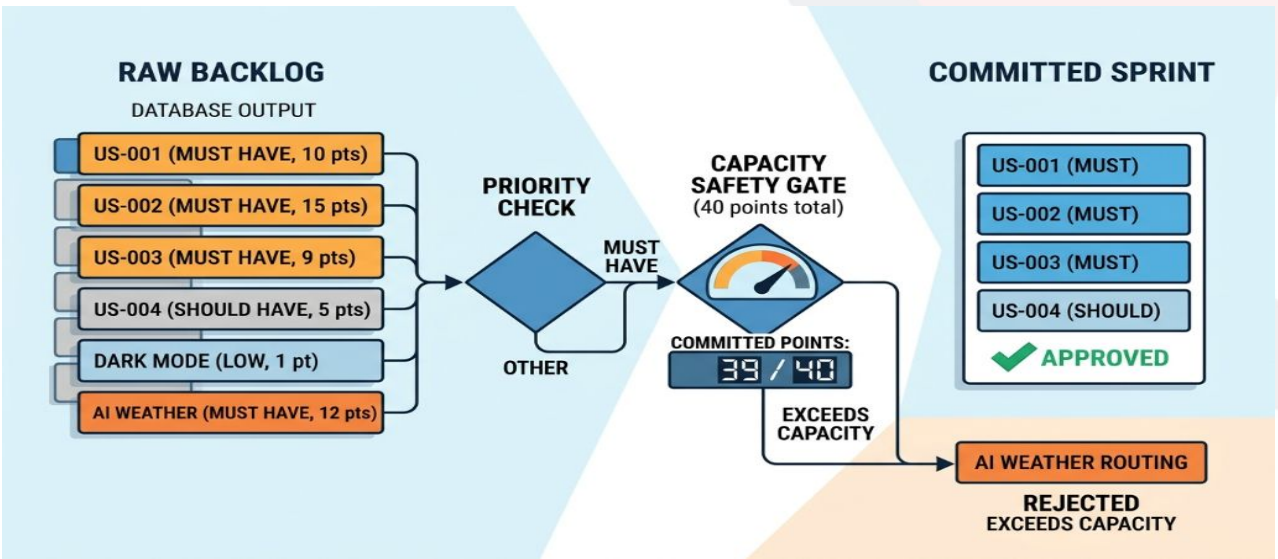
- `product_backlog`: This array simulates the raw database output from an Agile project management tool like Jira. It centralises the demands of the business.
- `if story['MoSCoW'] == 'Must Have':`: This algorithm enforces strict architectural discipline, parsing the array sequentially to prioritise the MVP above all else.
- `if (total_committed_points + story['StoryPoints']) <= capacity:`: This serves as the mathematical safety gate. If the business stakeholders demand more 'Must Have' features than the engineering team can physically deliver, the pipeline halts.

Execution Procedure

1. Open the Colab notebook.
2. Paste the Requirements Engineering code block into the first executable cell.
3. Click the 'Play' icon to process the Agile sprint calculation.

Expected Output

The console will systematically evaluate the backlog. It will commit `US-001`, `US-002`, and `US-003` because their combined points (34) fall under the 40-point capacity. It will seamlessly include the 'Should Have' `US-004` (5 points), bringing the total to 39 points. It will ignore the Dark Mode feature (as it is low priority and would exceed the remaining 1 point) and explicitly reject the AI Weather Routing. The engine will successfully approve the sprint.



© Copyright 2025 MIVA Open University All Rights Reserved

Session One Validation and Testing Protocol

To verify that the engine actively protects the engineering team from severe scope creep, a capacity breach test must be performed.

1. Create a new code cell below the initial setup.
2. Simulate a scenario where a massive new 'Must Have' feature is forced into the backlog and re-run the engine.

Python

```
# Validation Test: Simulating severe Scope Creep
toxic_backlog = product_backlog.copy()
toxic_backlog.append({'StoryID': 'US-007', 'Feature': 'Fleet Management
Dashboard', 'MoSCoW': 'Must Have', 'StoryPoints': 13})

print("\n--- Validation Run: Testing Scope Creep Rejection ---")
is_valid, failed_sprint = execute_moscow_sprint_planning(toxic_backlog,
sprint_capacity_points)

if not is_valid:
    print("\n[EXECUTIVE ALERT] The Sprint is mathematically
over-committed. Scope must be renegotiate before engineering begins.")
```

3. Execute the cell.

Validation Outcome: The test is definitively successful if the system commits the first three features, but upon reaching US-007, forcefully throws the [WARNING] Cannot commit... Sprint Capacity Exceeded! alert, returns False, and triggers the Executive Alert.

© Copyright 2025 MIVA Open University All Rights Reserved

Troubleshooting Guide

Error: KeyError 'MoSCoW'. Ensure that the dictionary keys in your backlog exactly match the capitalisation used in the function. Python is strictly case-sensitive.

Managerial Takeaway

Software projects do not fail at the end; they fail at the beginning. If the Requirements Gathering phase allows the business to dictate a scope that exceeds engineering capacity, the project is mathematically doomed. Implementing algorithmic capacity checks ensures the SDLC begins on a foundation of empirical reality.

© Copyright 2025 MIVA Open University All Rights Reserved